Synthetic Population Framework - Code documentation

Candice Baud
candice<dot>baud<at>epfl<dot>ch

June 2026

This document is a documentation to help users generate their own synthetic population. It is structured in the aim to be reused by other researchers. The first section details the installation of the code and the necessary packages. The second section introduces the basic elements of the framework to sample from priors (ie data-free); and the third section presents how to sample using data combined with prior specification. A final section presents how to run the provided examples.

This framework is presented more in details in the following papers :

> "Baud, C., & Bierlaire, M. (2026). A flexible and realistic synthetic panel population generation. In Proceedings of the 2026 ieee 29th international conference on intelligent transportation systems. (In press)"
>
> "Baud, C., & Bierlaire, M. (2026). From Priors to Data: A Flexible Bayesian Framework for Panel Synthetic Population Generation. In Proceedings of the 2026 hEART conference. (In press)"

Contents

1 Installation

Code availability

The code is available at `https://github.com/candicebaud/longitudinal_synthetic_population`. The folder `framework` contains the source code defining the framework. The folder `examples` contains examples on how to use the code. The folder `graphs` contains example notebooks to analyze the results and produce graphs.

Programming language

The project is implemented in Python. It has been tested successfully with Python 3.11.7 on SCITAS supercomputer and with Python 3.13.7 on a local machine. In practice, Python version 3.11.7 or newer is recommended.

The main external dependencies are `numpy`, `pandas`, and `pycddlib`.
A particular point of attention concerns `pycddlib`, which provides the module `cdd` used to compute polytope vertices from systems of linear inequalities. Two versions of this package are supported in the repository. On SCITAS, the code was run with an older `pycddlib` API (version 2.1.7), whereas the local version was run with the newer 3.x API. Since `pycddlib` 3.x introduced backward-incompatible changes in the construction of matrices and polyhedra, two implementations of the same function are provided in the repository, one for each API. Users should therefore select the version of the code that matches the installed `pycddlib` release. To install the dependencies, users may run:

```
pip install numpy pandas pycddlib
```

For environments using the older `pycddlib` API, the code version provided for SCITAS should be used. For environments using `pycddlib` 3.x, the local-machine version should be used.

Examples

In the folder `examples`, the user can find code examples to familiarize with the framework. The folder `prior_model` provides an example on how to sample a population from prior models.

Notations

The code files are denoted following a convention on the first word :

- `classes_` : correspond to the definition of storage classes
- `constraints_` : correspond to the classes defining the constraints
- `spec_` : correspond to classes to specify the models
- `har_` : correspond to hit-and-run algorithms
- `model_` : correspond to model definitions
- `sample_` : correspond to launch file to sample a model
- `run_` : correspond to runfiles in the case of a HPC usage

2 General framework to sample from priors

Probabilistic Model

The probabilistic model class defines the probabilistic model of the quantity that the user is trying to sample. It contains

- `sampler_full_conditional` : defines the exact sampler using the full conditionals. This sampler is used to perform an exact Gibbs update.
- `sampler_proposal` and `sampler_evaluation_proposal` that define a sampler function (a proposal), and the evaluation of the probability to have sampled the quantity. Those functions are used to perform a Metropolis-Hastings step with proposal.
- `sampler_initial` : This function defines the initial sampler, that is agnostic of any state.[1]

There are two methods in this class to make the definition easier :

- `.Gibbs` : the user only needs to define `sampler_initial` and `sampler_full_conditional`
- `.Proposal` : the user only needs to define `sampler_initial`, `sampler_proposal` and `sampler_evaluation_proposal`

AttributeSpec

This class defines everything related to an attribute in the model. The specification encodes the definition of the generation of values. The user needs to specify

- `name` : name of the attribute that will be used throughout the algorithm to refer to it
- `type` : *Single* or *Multiple*, depending on whether the attribute is associated with a single or multiple occurring event.
- `prob_model` : a probabilistic model as defined above, defining how the quantities should be sampled.

In this class are defined two methods :

- `SingleAttr` : that automatically sets the type to *Single*
- `MultipleAttr` : that automatically sets the type to *Multiple*

The sampler functions defined in the probabilistic model have different attributes whether the attribute is *Single* or *Multiple*. If the attribute is *Single*, the sampler is directly sampling one value and doesn't need more information. If the attribute is *Multiple*, the sampler needs to know which attribute is sampled. That means, the income for job spell i might not have the same definition of the income for job spell j at its core. The sampler can sample differently the attribute for different spells.

EventTemporalConstraintsSpec

This class defines the temporal constraints for an event. In particular lower and upper bounds on start ages and durations for both main and gap components of an event. The user must define

- `main_start_age_min` : The minimal age to enter start the main part of the event.
- `main_start_age_max` : The maximal age to enter the start of the main part of the event.
- `main_min_duration` : The minimal saturating duration of the main part of the event.

[1] As at the initialization, no state has been sampler yet

- `main_max_duration` : The maximal duration of the main part of the event.
- `gap_start_age_min` : The minimal age to enter start the gap part of the event.
- `gap_start_age_max` : The maximal age to enter the start of the gap part of the event.
- `gap_min_duration` : The minimal saturating duration of the gap part of the event.
- `gap_max_duration` : The maximal duration of the gap part of the event.
- `max_duration` : The total maximal duration of the event (main and gap).

EventSpec

EventSpec describes the specification of any event defined in the framework. The user needs to define :

- `name` : name of the Event that will be used throughout the algorithm to refer to it.
- `type` : to define the type of event between *NoEvent* or *Event*
- `max_count` : defines the maximal number of times an event can arrive ($\in \{0, M\}$). The value must be finite.
- *indicator probabilistic model* : probabilistic model to sample the indicators
- `attributes_spec` : a list of specifications of attributes associated to that event. The specification is done using `AttributeSpec`.
- `constraint_spec` : specification of the temporal constraints defining the event. The specification is done using `EventTemporalConstraintsSpec`

In this class are defined 3 methods :

- `NoEvent` : to define a *NoEvent*
- `SingleTimeEvent` : to create a single time event (the maximum number of occurrences is set to 1)
- `MultipleTimeEvent` : to create an event specification for events that can happen more than once (the maximum number of occurrences must be specified by the user)

DimensionSpec

This class defines the specification of the dimension. The user must specify

- `name` : name of the dimension that will be used throughout the algorithm to refer to it.
- `list_event_spec` : an ordered list containing the specification of the events of the dimension. The order is made based on the order of appearance of the events in life.
- `log_density` : in a case where the dimension is sampled "on its own", or other said independently from the other dimensions, the log density provided defines the target probability distribution of the durations of the events composing that dimension. This is optional to specify, but mandatory in a case of independent dimension sampling.

Remark : Due to the framework structure, note that there should always be a no event in each dimension in the first place.
In this class, one can sample the trajectory of the dimension by using two functions :

- `sample` : this function samples the trajectory using one chain.

- `sample_diagnostics` : this function samples many trajectories using multiple chains and returns the sampled values as well as diagnostics of convergence of the chains.

Both functions take into argument the following :

- `dob` : date of birth of the individual that is being sampled
- `lifespan` : lifespan of the individual that is being sampled
- `n_samples` : the number of individuals to sample[2]
- `n_gibbs` : the number of gibbs iterations to perform to sample the indicators
- `burnin` : the number of draws before attaining the convergence of the chain
- `seed` : for reproducible results, sets the random state of the algorithm

Additionally, `sample_diagnostics` has a parameter `n_chains` corresponding to the number of chains launched that are used to calculate the diagnostics.

DurationLinearConstraintsSpec

This class defines linear relationships between durations of different events, possibly across dimensions, of the form

$$\text{path}_1 \leq \sum_k a_k \cdot \text{path}_k + b \tag{1}$$

where a path is simply an access to the duration of an event to locate it and affect the constraint. In order to specify the path, one must use the class `DurationPath` in which the user specifies

- `dimension_name` : the name of the dimension in which the event belongs; as defined in `DimensionSpec`
- `event_name` : the name of the event of interest to constrain; as defined in `EventSpec`
- `event_number` : for multiple occurring events, the name is not enough to locate which of the spell is concerned. Therefore, the user must also specify which of the spell must be constrained.
- `duration_type` : to specify if the main or the gap part of the event is concerned
- `no_event_bool` : 1 if the event is a *NoEvent*

These paths can be specified more easily using the following methods

- `NoEvent`
- `SingleTimeEvent`
- `MultipleTimeEvent`

For `DurationLinearConstraintsSpec`, we need to specify the following :

- `path_1` : the path to the duration that is constrained; using `DurationPath`
- `list_paths` : the paths of the other durations that constrain it (the path_k); using as well `DurationPath`
- `list_a`: the a values corresponding to the linear coefficients in 1
- `eq_bool` : a 1 or 0 value which states if the constraint is an inequality or an equality. Bool = 0 means the inequality as in 1. Bool = 1 means we define an equality instead.

[2]Note that all sampled individuals have the same `dob, lifespan`

IndividualSpec

This class defines the full generative specification for an Individual, including dimension specifications, inter-event constraints, and the joint distribution governing durations.

The `IndividualSpec` class must be specified using :

- `id` : id of the individual to identify it.
- `list_dim_spec`: the list of dimension specifications that will enable to generate the life trajectories of the individual. The list of dimensions in unordered except for the first dimension corresponding to *Existence*.
- `joint_dist_duration` : the joint log density of the durations (for all dimensions)
- `list_inter_constraint_spec` : the list of constraints between events; specified by `DurationLinearConstraintsSpec`

From this class, one can sample an individual using `sample` and indicating the `dimension_mode` as *joint* for sampling dimensions altogether, or *independent* for sampling each dimension independently of the other, or *automatic* in which the algorithm automatically samples the durations based on the constraint structure. The user must also specify parameters for the sampling such as the burnin phase and the thin, just like for the dimensions.

PopulationSpec

The class defining the population specification. For now, a population is a collection of independent individuals. Therefore, the user must specify

- `size` : how many individuals to sample in the population
- `list_dim_spec`: the list of dimension specifications
- `joint_dist_duration` : the joint log density of the durations (for all dimensions)
- `list_inter_constraint_spec` : the list of constraints between events specified by `DurationLinearConstraintsSpec`

The definition is based exactly on the same objects as the individual definition as the population is a collection of independent individuals following the same governing probabilistic model.

Similarly to `IndividualSpec`, the class has a function `sample` to sample a population. The same arguments can be specified. Additionally, the function `sample_parallel_to_disk` implements a parallelized generation of individuals to make the process faster.

The case of Existence

The Existence dimension is mandatory as it defines temporal bounds on the individual's life and thus determines the rest of the other dimensions trajectories. The Existence dimension is structurally different than the other dimensions, hence the specific class to define it.

BirthConstraintSpec

Specification of temporal constraints for the birth event. This class defines admissible bounds on the birth time and lifespan.

The user must provide

- `start_high` : the highest date of birth possible (in years)
- `start_low` : the lowest date of birth possible (in years)

- `duration_high` : the highest lifespan possible
- `warmup_time` : this object defines a *warmup time* from which individual start to be born, before the lowest possible date of birth. Indeed, if the user is interested in a population in [x,y], specifying a lowest date of birth of interest x will not yield a correct population at time x. At time x, individuals can only be of age equal to 0. in x + u, individuals can only at maximal be age u. Therefore, to avoid this edge effect, the user can define a warmup time[3] w and the algorithm will generate individuals from x-w and only keep the ones alive at x, hence defining at x a valid population; and avoiding over-sampling outside of the bounds.

ExistenceEventSpec

The class defines the event in Existence, to which we refer as *Birth*. To do so, the user must provide

- `attributes_spec` : a list of attributes specification if any attributes are attached to this event, using the class `AttributesSpec`
- `log_density` : the log density of the date of birth and lifespan variables
- `constraints` : the constraints defined using `BirthConstraintSpec`

ExistenceDimensionSpec

This class defines the specification of the Existence Dimension. The class only contains one event, the *Birth* event and is sampled independently from the other dimensions.

Storage of the results

The obtained results are stored using different classes for the different objects.

Attribute

Each attribute contains a `name` and a `type` corresponding to the ones defined in the `AttributeSpec`, and a `value` corresponding to its sampled value.

Event

Each event contains a `name` and a `type` corresponding to the ones defined in `EventSpec`, as well as a `main_start_date`, `gap_start_date`, `duration`, `main_duration`, `gap_duration`, extracted from the sampled durations; and finally `attributes` which is a list of attributes of type `Attribute` defined above.

Trajectory

Each trajectory contains a `dim_name` corresponding to the dimension name associated, defined by `DimensionSpec`. It also contains a `list_events` containing the events as `Event` occurring in that dimension, and `list_indic` containing 0-1 values corresponding to indicators of each event occurring or not.

Individual

Each individual contains a `id` corresponding to the identifier defined in `IndividualSpec`; and `trajectories` as defined above as `Trajectory`.

Additionally, the `Individual` class contains a `.to_dataframe` function that enables to create a dataframe containing the life characteristics (dimensions, event, durations, attributes) of the individual from its sampled trajectories.

[3]To have a realistic population at time x, one must at least define a warmup equal to the maximal lifespan.

Population

Each population contains a `list_individuals` corresponding to the sampled individuals.

It as well contains a `.to_dataframe` function to create a dataframe with the lives (dimensions, event, durations, attributes) of all individuals in the sampled population.

3 Examples

This section describes how to run the examples provided in the github folder.
A general note is that the `framework` folder contains the code to be used with `pycddlib 3.x` or more. If you are using an older version, use the different files in the `hpc_friendly` folder.

Priors

An example to sample from the priors is available in the folder `examples/prior_models`. The files in the folder are :

- `model_priors` : a file defining the probabilistic model used for the generation of results in the papers mentioned above
- `sample_priors` : a file to run the code. It is calibrated for a HPC. In order to run it on a personal laptop, calibrate the parameter `n_workers` appropriately to correspond to the number of parallel jobs you want to run.
- `run_priors` : a run file that can be used to run the experiments on a HPC.

Note that if you want to run the experiments on your personal laptop using parallel processing, it is advised to launch it from a separate file like `sample_priors`.

Once you start to run the code, each individual, once generated, will be saved as a csv file in a folder of the name specified by `output_dir`. If you want to plot the same graphs as provided in the paper, use the folder `graphs/prior_graphs`. A separate subsection details its content.

Priors graphs

Once you have generated the individuals, merge all the individuals dataframe into one. If you have not generated your own data, you can still recreate the plots on a subsample of 2000 individuals that is provided (instead of 50000 in the papers)[4]. The provided dataframe is the file called `prior_2000indiv.csv`. A file `graphs_style` enables to setup the style of the graphs. Finally, the notebook `prior_graphs` enables to plot the desired graphs.

[4]Using the restricted dataframe, one should expect the plots to be more noisy as the sample is much smaller.